

WEEK 1 LEARNING GUIDE

AWS CLOUD FOUNDATIONS & INFRASTRUCTURE AS CODE

1. INTRODUCTION

The first week of the internship focused on building a strong foundation in AWS Cloud and Infrastructure as Code (IaC). Instead of starting with serverless computing or backend development, the objective was to understand how cloud infrastructure is provisioned, managed, and secured before building applications on top of it.

The week's project involved creating a static website hosting platform on AWS using Terraform. Although the final application was relatively simple, the implementation introduced several fundamental cloud concepts including Infrastructure as Code, Amazon S3, IAM, CloudFront, GitHub Actions, and AWS CLI. Along the way, the project evolved from a basic S3-hosted website into a more production-oriented architecture using a private S3 bucket behind CloudFront with Origin Access Control (OAC).

By the end of the week, I had a much better understanding of how AWS services work together and how cloud infrastructure can be automated, secured, and managed using code rather than manual configuration.

2. CORE CONCEPTS LEARNED

INFRASTRUCTURE AS CODE (TERRAFORM)

One of the biggest concepts introduced this week was **Infrastructure as Code (IaC)**. Before learning Terraform, I mainly associated cloud infrastructure with creating resources manually through the AWS Management Console. Terraform introduced a completely different approach where infrastructure is written as code and deployed automatically.

What I understood

- Infrastructure can be described declaratively instead of configuring resources manually.
- The same configuration can be deployed multiple times with consistent results.
- Infrastructure changes can be version controlled just like application source code.
- Terraform determines what needs to be created, updated, or deleted by comparing the current infrastructure with the desired configuration.

A typical Terraform workflow consists of four commands:

```
terraform init
terraform plan
terraform apply
terraform destroy
```

Each command serves a specific purpose:

- **terraform init** initializes the working directory and downloads the required providers.
- **terraform plan** previews the infrastructure changes before deployment.
- **terraform apply** provisions or updates the infrastructure.
- **terraform destroy** removes all resources managed by Terraform.

This workflow encouraged a much safer deployment process compared to directly creating resources in the AWS Console.

Important concepts learned

Providers

Providers allow Terraform to communicate with cloud platforms.

For AWS:

```
provider "aws" {
  region = "ap-south-1"
}
```

Resources

Every AWS service is represented as a resource.

Example:

```
resource "aws_s3_bucket" "website" {
  bucket = var.bucket_name
}
```

Terraform keeps track of every resource it creates.

Variables

Variables make infrastructure reusable by avoiding hardcoded values.

```
variable "bucket_name" {}
```

Instead of editing the code for every deployment, only the variable values need to change.

Outputs

Outputs expose useful information after deployment.

Example:

```
output "bucket_name" {  
  value = aws_s3_bucket.website.bucket  
}
```

These outputs can also be referenced by other modules or CI/CD workflows.

Modules

Initially, the infrastructure could have been written in a single file. However, as more AWS resources were added, it became clear that organizing everything in one place would quickly become difficult.

Terraform modules solve this problem by grouping related resources into reusable components.

Example:

```
module "website_bucket" {  
  source      = "../modules/s3_bucket"  
  bucket_name = var.bucket_name  
}
```

This made the project:

- Easier to organize.
- Easier to reuse.
- Easier to extend in future weeks.

Terraform State

One concept I had not encountered before was the Terraform state file.

Terraform stores information about all managed resources inside:

terraform.tfstate

This file allows Terraform to understand the current infrastructure before applying future changes.

During implementation, I also learned an important best practice:

- **terraform.tfstate** should never be committed to Git.
- The **.terraform/** directory should also be ignored.

The project included a **.gitignore** similar to:

```
.terraform/  
*.tfstate  
*.tfstate.*
```

Later, I also learned that production projects usually store the Terraform state remotely in an S3 bucket instead of keeping it locally, allowing multiple developers to collaborate safely.

Application in this project

Terraform was used to provision almost the entire infrastructure required for the project, including:

- Amazon S3 bucket
- Bucket versioning
- Server-side encryption
- Lifecycle rules
- Public access block
- Bucket policies
- CloudFront distribution
- Origin Access Control (OAC)

Using Terraform instead of manual configuration made the deployment process repeatable and significantly reduced the possibility of configuration errors.

AMAZON S3 (SIMPLE STORAGE SERVICE)

Amazon S3 was the primary AWS service used throughout this week's project. Initially, I understood S3 as a cloud storage service used for storing files and hosting static websites. However, while implementing the project and learning its advanced features, I realized that S3 is much more than simple cloud storage. It is a highly durable object storage service that serves as the storage layer for many AWS applications.

One important concept I learned early on was that **S3 is an object storage service, not a traditional file system**. Instead of organizing files into actual folders, S3 stores every file as an object inside a bucket. The folder structure we see in the AWS Console is only part of the object's key.

For example,

documents/resume.pdf

is stored as a single object with the key **documents/resume.pdf**, not as a file inside a real folder named **documents**.

Understanding this difference made it easier to understand how S3 manages objects internally and why it can scale to billions of objects.

Buckets and Objects

Every object in S3 is stored inside a bucket.

During this project, I learned:

- Buckets are globally unique across AWS.
- Every object belongs to exactly one bucket.
- Objects contain both the file data and metadata.
- Each object is identified using its key.

For this project, Terraform was used to create the bucket instead of manually creating it through the AWS Console.

```
resource "aws_s3_bucket" "website_bucket" {  
  bucket = var.bucket_name  
}
```

Using Terraform made bucket creation repeatable and ensured that the infrastructure remained consistent across deployments.

Bucket Versioning

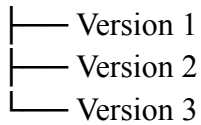
One of the first advanced S3 features I explored was **Versioning**.

Without versioning, uploading a file with the same name simply replaces the previous file.

```
index.html  
|  
New Upload  
|  
Old File Lost
```

With versioning enabled:

index.html



Previous versions remain available and can be restored if required.

Terraform configuration:

```
resource "aws_s3_bucket_versioning" "website_versioning" {
  bucket = aws_s3_bucket.website_bucket.id

  versioning_configuration {
    status = "Enabled"
  }
}
```

Why it matters

- Protects against accidental overwrites.
- Makes rollback possible.
- Preserves historical versions of files.
- Useful for recovery after accidental deletion.

While implementing this, I also learned that every version is stored as a complete object, which means versioning improves reliability but also increases storage usage.

Server-Side Encryption (SSE)

Before this week, I had very little understanding of data encryption. Learning about **Server-Side Encryption (SSE)** helped me understand how AWS protects data while it is stored inside S3.

For this project, **SSE-S3** was used.

```
server_side_encryption_configuration {
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}
```

What I understood

AWS supports multiple encryption options:

Type	Description
SSE-S3	AWS manages both the encryption and the keys.
SSE-KMS	AWS KMS manages encryption keys with additional access control and auditing.
SSE-C	The customer provides the encryption keys.

For this project, SSE-S3 was sufficient because it provides automatic encryption with minimal configuration.

Another important distinction I learned was:

- **Encryption protects data at rest.**
- **HTTPS protects data in transit.**

Initially, I assumed these were the same thing, but they solve two completely different security problems.

Lifecycle Rules

Another feature that I found interesting was **Lifecycle Management**.

As applications grow, storing every object indefinitely becomes expensive. Lifecycle rules allow S3 to automatically move or delete objects based on predefined conditions.

Typical lifecycle actions include:

- Moving old objects to cheaper storage classes.
- Deleting objects after a certain period.
- Cleaning up incomplete multipart uploads.

Example:

Upload Object

|

30 Days

|

Move to Glacier

|

365 Days

|

Delete Object

For this project, lifecycle rules were configured through Terraform to better understand how automated storage management works.

The biggest takeaway here was that cloud storage isn't only about storing data—it also involves managing storage costs over time.

Bucket Policies vs ACLs

While configuring bucket permissions, I initially thought **Bucket Policies** and **ACLs** served the same purpose.

After studying both, I learned that modern AWS architectures generally prefer **Bucket Policies**.

Bucket Policies	ACLs
JSON-based	Permission lists
More flexible	Limited functionality
Easier to manage	Considered legacy
Preferred by AWS	Mostly avoided today

The project used bucket policies throughout the implementation.

This became especially important later when configuring CloudFront to access a private S3 bucket using Origin Access Control.

Presigned URLs

One of the most practical concepts introduced this week was **Presigned URLs**.

Normally, if an S3 bucket is private, users cannot upload or download objects directly.

Presigned URLs solve this problem by generating a temporary URL that allows a specific operation for a limited period.

Typical workflow:

Client

| Request Upload URL



Backend

| Generates Presigned URL



Amazon S3



Client uploads directly

This approach provides several advantages:

- Bucket remains private.
- AWS credentials are never exposed.
- Backend does not need to handle file uploads.
- Uploads are faster because files go directly to S3.

Generating a PUT URL:

```
url = s3.generate_presigned_url(  
    "put_object",  
    Params={  
        "Bucket": bucket_name,  
        "Key": object_name  
    },  
    ExpiresIn=3600  
)
```

Challenge encountered

While testing the implementation, the generated presigned URL worked correctly using Python's **requests** library but consistently failed in Postman with a **SignatureDoesNotMatch** error.

After debugging the requests, I found that Postman automatically added a different **Content-Type** header than the one used while generating the URL.

The solution was to:

- Detect the file's MIME type using Python's **mimetypes** module.
- Include the **ContentType** while generating the presigned URL.
- Send the exact same **Content-Type** header during the upload.

```
content_type, _ = mimetypes.guess_type(file_path)
```

This debugging session helped me understand that AWS signs the **entire request**, not just the URL. Even a small mismatch in signed headers causes the request to be rejected.

Multipart Upload

Uploading large files as a single request can be unreliable. If the upload fails near completion, the entire file has to be uploaded again.

Multipart Upload solves this problem by splitting the file into smaller parts.

Large File

|

Split into Parts

|

Part 1

Part 2

Part 3

...

|

Upload Independently

|

Amazon S3 Reassembles Object

Key concepts I learned:

- Minimum part size: **5 MB** (except the last part).
- Failed parts can be retried individually.
- Parts can be uploaded in parallel.
- Maximum object size supported is **5 TB**.

Although boto3 can automatically perform multipart uploads, implementing the manual workflow helped me understand how enterprise applications handle large uploads efficiently.

Application in this project

Amazon S3 formed the storage layer for the entire project.

The final implementation included:

- Provisioning the bucket using Terraform.
- Versioning for object recovery.
- SSE-S3 encryption.
- Lifecycle rules.
- Bucket policies.
- Public access blocks.
- Presigned upload and download URLs.
- Multipart upload implementation.
- CloudFront integration using a private S3 origin.

By the end of the week, my understanding of S3 had evolved from "a place to store files" to recognizing it as one of the core services used in secure, scalable, and production-ready AWS architectures.

IDENTITY AND ACCESS MANAGEMENT (IAM)

Before this week, I only knew that IAM was used to create AWS users. While setting up AWS CLI and Terraform, I understood that IAM is much more than user management—it is the service responsible for controlling **who can access AWS resources and what actions they are allowed to perform**.

One of the most important concepts I learned was the difference between **authentication** and **authorization**.

- **Authentication** verifies *who* the user is.
- **Authorization** determines *what* the user is allowed to do.

AWS IAM provides this authorization using policies.

Example IAM policy:

```
{  
  "Effect": "Allow",  
  "Action": [  
    "s3:GetObject"
```

```
],  
  "Resource": "*" }  
}
```

Important concepts learned

IAM Users

- Represent individual AWS users.
- Used for programmatic and console access.
- Can be assigned permissions directly or through groups.

IAM Policies

- JSON documents defining permissions.
- Can Allow or Deny specific actions.
- Can be attached to users, groups, or roles.

IAM Roles

Although roles were not extensively used during this week's implementation, I learned that roles allow AWS services to securely access other AWS services without requiring long-term credentials.

This becomes especially important in serverless applications, which will be explored in later weeks.

AWS CLI Configuration

To allow Terraform to communicate with AWS, the AWS CLI was configured using IAM credentials.

aws configure

This stores:

- Access Key ID
- Secret Access Key
- Default Region
- Output Format

Terraform automatically uses these credentials while provisioning infrastructure.

Principle of Least Privilege

One important security practice introduced this week was the **Principle of Least Privilege**.

Instead of giving every user full administrative permissions, access should be limited to only the actions required.

For example:

- Terraform only needs permissions to create the resources being managed.
- CloudFront only needs permission to read objects from S3.
- Users should never receive unnecessary permissions.

This principle appeared repeatedly throughout the week's implementation and helped me understand that security is largely based on restricting access rather than simply protecting resources.

Application in this project

IAM was used for:

- Configuring AWS CLI authentication.
- Allowing Terraform to provision AWS resources.
- Managing permissions between CloudFront and S3.
- Understanding how AWS services securely communicate with each other.

Although IAM configuration was relatively small in this project, it became clear that every AWS service depends on IAM in some way.

AMAZON CLOUDFRONT

CloudFront was introduced after learning the basics of Amazon S3. Initially, I assumed that hosting a static website directly from an S3 bucket would be sufficient. However, learning about CloudFront demonstrated why production systems usually place a Content Delivery Network (CDN) in front of storage services.

CloudFront improves both **performance** and **security** by caching content at AWS edge locations while preventing users from directly accessing the storage bucket.

What I understood

CloudFront is AWS's Content Delivery Network (CDN).

Instead of every request reaching the S3 bucket:

User

|
▼
Amazon S3

requests are routed through CloudFront.

User
|
▼
CloudFront
|
▼
Amazon S3

CloudFront stores cached copies of frequently accessed content at edge locations around the world, reducing latency and improving load times.

Important concepts learned

Origins

The origin is the service from which CloudFront retrieves content.

For this project:

- Origin = Amazon S3 Bucket

Caching

When a user requests an object:

- First request → CloudFront fetches it from S3.
- Later requests → Served directly from CloudFront cache.

This reduces:

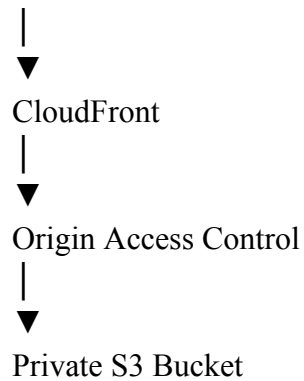
- Response time
- Load on the origin
- Bandwidth usage

Origin Access Control (OAC)

One of the biggest architectural improvements during this project was replacing a public S3 bucket with a **private bucket protected by CloudFront OAC**.

Final architecture:

User



This means:

- Users cannot access S3 directly.
- Only CloudFront is allowed to retrieve objects.
- The bucket remains private.

This was one of the most important production-oriented concepts I learned during the week.

HTTPS

CloudFront automatically provides HTTPS using its default certificate.

Even without purchasing a custom domain, the website could be securely accessed through:

`https://xxxxxxx.cloudfront.net`

This helped me understand that HTTPS is not tied to owning a custom domain.

Cache Invalidation

After updating files inside S3, CloudFront continued serving the old cached version. Initially, this appeared to be a deployment issue.

I later learned that CloudFront caches objects until either:

- the cache expires, or
- an invalidation request is sent.

Example:

```
aws cloudfront create-invalidation \  
  --distribution-id XXXXX \  
  --paths "/*"
```

This demonstrated that content delivery networks introduce another layer that must be considered during deployments.

Application in this project

CloudFront was configured to:

- Deliver website assets globally.
- Cache static content.
- Redirect HTTP requests to HTTPS.
- Compress responses.
- Access a private S3 bucket using OAC.
- Improve security by hiding the storage layer from users.

The transition from a public S3 website to a private S3 bucket behind CloudFront was one of the biggest architectural improvements made during the project.

GITHUB ACTIONS (CI/CD)

The final concept introduced this week was **Continuous Integration and Continuous Deployment (CI/CD)** using GitHub Actions.

Initially, every change required manually uploading files to S3 and invalidating the CloudFront cache. While this works for small projects, it quickly becomes repetitive and error-prone.

GitHub Actions automated this workflow.

What I understood

A GitHub Actions workflow is triggered whenever specific events occur, such as pushing code to a repository.

Typical workflow:

Git Push

|

GitHub Actions

|

Upload Website

|

Invalidate CloudFront Cache

|

Deployment Complete

Instead of manually performing multiple deployment steps, the workflow automatically handled them.

Benefits

- Reduced manual work.
- Consistent deployments.
- Faster updates.
- Lower chance of deployment mistakes.

Application in this project

GitHub Actions was used to:

- Deploy website files to Amazon S3.
- Automatically invalidate the CloudFront cache.
- Keep the deployment process consistent.

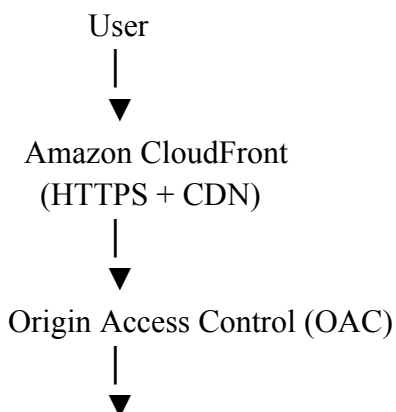
Although the workflow itself was relatively small, it introduced an important DevOps practice that will become increasingly useful in larger projects.

3. PROJECT IMPLEMENTATION

The concepts learned throughout the week were applied by building a static website hosting platform on AWS using Terraform. While the application itself was simple, the project focused more on building the supporting cloud infrastructure using modern AWS practices.

FINAL ARCHITECTURE

The final implementation followed the architecture shown below:



Private Amazon S3 Bucket



Static Website Assets

Instead of exposing the S3 bucket directly to the internet, CloudFront acted as the public entry point while the bucket remained private.

This approach provided:

- Improved security
- Global content delivery
- HTTPS by default
- Better caching performance
- Better scalability

Compared to the original public bucket approach, this architecture is much closer to what is used in production.

INFRASTRUCTURE PROVISIONING

Terraform was responsible for provisioning almost the entire infrastructure.

The project was organized using reusable modules instead of placing every resource inside a single Terraform file.

Project structure:

```
project/
├── main.tf
├── variables.tf
├── outputs.tf
├── terraform.tfvars
└── modules/
    └── s3_bucket/
        ├── main.tf
        ├── variables.tf
        └── outputs.tf
```

This organization made the infrastructure:

- Easier to understand
- Easier to maintain
- Reusable in future projects

One thing I appreciated during implementation was that adding new resources became much easier once the infrastructure was modularized.

CONFIGURING AMAZON S3

The S3 bucket was configured with several production-oriented features instead of only creating an empty bucket.

The implementation included:

- Versioning
- Server-side encryption
- Lifecycle policies
- Public access block
- Bucket policies

Example:

```
resource "aws_s3_bucket_versioning" "versioning" {  
  
    bucket = aws_s3_bucket.website.id  
  
    versioning_configuration {  
        status = "Enabled"  
    }  
  
}
```

Initially, hosting the website using S3 Static Website Hosting seemed like the obvious solution. However, after learning about CloudFront and OAC, I realized that exposing the storage bucket publicly is generally avoided in production.

The implementation was therefore updated to use a private bucket behind CloudFront instead.

This was probably the biggest architectural improvement made during the project.

CONFIGURING CLOUDFRONT

CloudFront was placed in front of the S3 bucket to provide secure content delivery.

Important configurations included:

- Origin Access Control
- HTTPS redirection
- Compression
- Default cache behavior

- CloudFront default certificate

One challenge encountered during this stage was understanding why changes made to the website did not immediately appear.

Initially, it seemed like Terraform had failed to upload the updated files.

After investigating further, I learned that CloudFront was still serving cached content.

The issue was resolved by creating a cache invalidation.

```
aws cloudfront create-invalidation \  
  --distribution-id XXXXX \  
  --paths "/*"
```

This was also the first time I understood why deployment pipelines often include automatic cache invalidation.

WORKING WITH PRESIGNED URLS

Python and boto3 were used to generate presigned URLs for uploading and downloading objects.

Example:

```
url = s3.generate_presigned_url(  
    "put_object",  
    Params={  
        "Bucket": bucket_name,  
        "Key": object_name  
    }  
)
```

The implementation worked correctly when tested using Python.

However, testing the same URL in Postman consistently resulted in:

SignatureDoesNotMatch

After comparing both requests, I found that Postman automatically added a different **Content-Type** header than the one used while generating the URL.

The solution was to:

- Detect the MIME type automatically.

```
content_type, _ = mimetypes.guess_type(file_path)
```

- Include the content type while generating the URL.
- Send the same header during upload.

Although the issue was relatively small, debugging it helped me understand how AWS request signing works and why even small header differences can invalidate a signed request.

MULTIPART UPLOAD

To understand how S3 handles large files, multipart upload was implemented using boto3.

Instead of uploading one large file:

Large File

|

Upload

the file is divided into multiple parts.

Large File

|

Split

|

Part 1

Part 2

Part 3

|

Upload Separately

|

S3 Combines Parts

Some important observations were:

- Failed parts can be retried individually.
- Multiple parts can be uploaded simultaneously.
- Large uploads become much more reliable.

I also learned that boto3 automatically performs multipart uploads for large files, but implementing it manually helped me understand what happens internally.

CI/CD AUTOMATION

Towards the end of the project, GitHub Actions was introduced to automate deployment.

Instead of manually:

- Uploading website files
- Updating S3
- Invalidating CloudFront

a Git push automatically triggered the deployment pipeline.

Workflow:

Git Push

|

GitHub Actions

|

Upload Files

|

CloudFront Invalidation

|

Website Updated

Although this automation was relatively simple, it demonstrated how repetitive deployment tasks can be automated using CI/CD pipelines.

4. KEY LEARNINGS & TAKEAWAYS

Week 1 introduced many AWS services, but the most valuable learning was understanding how these services work together rather than learning them individually.

Some of the key takeaways from this week are:

- Infrastructure can be managed just like application code using Terraform.
- AWS services are designed to work together, with each service handling a specific responsibility.
- Amazon S3 is much more than cloud storage; it provides features such as versioning, encryption, lifecycle management, and secure object access.
- CloudFront improves both performance and security by acting as a caching layer in front of private storage.
- IAM permissions should always follow the Principle of Least Privilege.
- CI/CD automation reduces repetitive manual deployment steps and improves consistency.
- Debugging cloud applications often requires understanding how AWS services communicate rather than only examining application code.
- Production architectures are usually different from beginner examples. During this project, moving from a public S3 bucket to a private S3 bucket protected by CloudFront and OAC clearly demonstrated this difference.

Overall, this week provided a solid foundation for the rest of the internship. Rather than only learning individual AWS services, I developed a better understanding of cloud architecture, Infrastructure as Code, security, and deployment automation. These concepts will be directly applicable in the following weeks as the projects evolve from static website hosting to serverless APIs, event-driven architectures, and more production-oriented cloud applications.